PAC-MAN ARCADE: COMPLETE CODE & ARCHITECTURE GUIDE

Comprehensive line-by-line explanation of pacman.py: Functions, Loops, Conditions, Data Structures, and Classical AI (A* Algorithm)

Project: Classical AI Pac-Man | **Framework:** Python Standard Library (Curses, Heapq, Time) | **Semester:** 5th Semester (AI)

1. High-Level Architecture & Lifecycle Flow

The application is structured into four cooperative layers: **Presentation (Curses TUI)**, **Game Loop & State Machine (Game class)**, **Classical AI (A* Pathfinding & Heuristics)**, and **Menu Navigation Screens**. The execution lifecycle proceeds as follows:

- **Entry Point:** curses.wrapper(main_menu) safely initializes the terminal, configures raw input mode, hides the cursor, initializes 15 color pairs, and handles graceful screen restoration upon exit.
- **Main Menu Loop:** An interactive loop displays the retro ASCII logo and handles arrow navigation across 4 options: Play, How to Play, Change Difficulty, and Quit.
- **Game Initialization:** When Play is selected, Game(stdscr, difficulty) is instantiated. It constructs the 23×55 arcade labyrinth, calculates total pellets, positions Pac-Man at (1,1), and spawns ghosts inside the central Ghost House.
- **Controls Pop-up:** A modal dialog presents rules, 3-lives mechanics, and keybindings before gameplay starts.
- **Core Tick Loop (run()):** Runs at ~50 FPS (20ms idle delay). In each tick, it: (1) moves ghosts if delta-time exceeds ghost speed, (2) tests collisions with Pac-Man, (3) tests the victory condition (all pellets eaten), (4) renders the active frame via draw(), and (5) samples non-blocking player input with single-press throttling.
- **End-Game Action:** Winning or losing displays an end dialog with explicit choices: press **[Q]** to terminate the game completely, or press **[ENTER]** to return to the Main Menu.

2. Global Constants, Symbols & Color System (Lines 1–82)

Component / Symbol	Lines	Role, Purpose & Logic
curses, time, heapq	1–6	Standard library dependencies. curses drives terminal graphics; time manages delta-time movement ticks; heapq implements the binary min-heap for A* Search ($O(\log V)$ priority queue).
WALL, EMPTY, PELLET, POWER_PELLET	8–11	Tile glyphs: '#' for impassable walls, ' ' for open corridors, '*' for standard 10-point food stars, and 'O' for 50-point power energizers.
PACMAN_SPRITES	14–19	Dictionary mapping directional vectors (dy, dx) to 2-frame animation pairs: Up ('^', 'A'), Down ('v', 'V'), Left ('<', 'D'), Right ('>', 'C'). Alternated via chomp_frame % 2.
GHOST_DEFS	22–29	List of 6 ghost definitions: Blinky ('M', Red), Pinky ('W', Magenta), Inky ('&', Cyan), Clyde ('@', Yellow), Shadow ('$', Green), Spooky ('%', White).
DIFFICULTY_SETTINGS	40–44	Configuration table: **Easy** (2 ghosts, 0.50s delay), **Medium** (4 ghosts, 0.35s delay), and **Hard** (6 ghosts, 0.22s delay). Sets the pacing and hunting radius.
init_all_colors()	46–82	Configures 15 terminal color pairs using curses.init_pair(). Establishes electric neon blue walls (Pair 8), golden yellow pellets (Pair 9), and vibrant ghost identities.

3. The Game Class: Initialization & Setup (Lines 84–174)

Game.__init__(self, stdscr, difficulty='Medium') (Lines 85–110):
Initializes core match variables: pacman_pos=(1,1), pacman_dir=(0,1), score=0, lives=3, and paused=False. Loads difficulty configuration and establishes movement timestamps (last_ghost_move and player_move_delay=0.12s) to throttle input and prevent typematic key repeat.

load_classic_arcade_maze(self) (Lines 111–156):
Loads a full 23-row by 55-column arcade labyrinth featuring outer barriers, symmetrical internal T-bars, a central **Ghost House** (rows 11–15, cols 22–32), and horizontal wrap-around side avenues on row 10. Converts string rows into mutable character lists (self.map), clears Pac-Man's starting cell at (1,1), and calculates initial pellet inventory: self.total_pellets = sum(row.count('*') + row.count('O')).

place_entities(self) (Lines 157–174):
Positions Pac-Man at cell (1,1) and distributes the active ghosts across designated spawn cells inside the central Ghost House [(13,25), (13,27), (13,29), (12,27), (11,27), (15,27)]. Each ghost dictionary stores mutable coordinates pos: [y, x] and metadata def: g_def. Used both on match startup and upon respawning after losing a life.

4. Rendering Engine & Visual Presentation (Lines 175–290)

draw(self) (Lines 175–248):
Coordinates frame generation without terminal flickering:

- **Centering Math:** Queries window size (h, w = getmaxyx()) and calculates offset_y = max(1, (h - maze_rows)//2) and offset_x = max(1, (w - maze_cols)//2) to perfectly center the maze.
- **Top HUD Anchor (Row 0):** Displays score, lives formatted with retro (C) (C) (C) icons, current difficulty, and remaining pellets in warm amber (color_pair(11)). Anchoring to row 0 ensures the HUD never scrolls off-screen.
- **Grid Rendering Loop:** Iterates over every tile (y, x). Pac-Man is drawn first using PACMAN_SPRITES with alternating open/chomp frames. Ghosts are drawn next with individual color attributes. Impassable walls are rendered as neon blue '#', food as golden '*', and power pellets as bold yellow 'O'.
- **Overlays & Refresh:** Adds the bottom controls helper line, displays the pause modal if self.paused is true, and flushes output via self.stdscr.refresh().

draw_popup(self, title, lines) (Lines 249–290):
Renders reusable modal dialog boxes (controls popup, life lost, pause, game over). Computes required width and height, clears the bounding box with solid dark background tiles, draws double-line ASCII borders (+===...===+, |), centers the title in cyan/gold, and renders body lines with context-sensitive color highlights.

5. Pac-Man Mechanics & Wrap-Around Tunnels (Lines 291–315)

move_pacman(self, direction): Executed when a valid directional key is accepted.

- **Vector Calculation:** Translates key code through DIRS to retrieve delta (dy, dx), updates self.pacman_dir, and increments chomp_frame.
- **Horizontal Wrap-Around Tunnels:** If Pac-Man exits the left boundary (nx < 0), his position is wrapped to the far right (nx = maze_cols - 1); if exiting the right (nx >= maze_cols), he wraps to column 0. This recreates the authentic arcade side escape routes.
- **Wall Collision & Pellet Consumption:** Verifies map[ny][nx] != WALL. When moving into a pellet ('*'), the tile is cleared to ' ', score increases by 10, and total_pellets decrements by 1. Power pellets ('O') award 50 points.

6. Classical AI Engine: A* Pathfinding & Ghost Personalities (Lines 316–402)

The ghost movement system is powered by textbook **Classical Artificial Intelligence**: the **A* (A-Star) Search Algorithm** coupled with an **admissible Manhattan heuristic**.

THE A* EVALUATION FUNCTION & ADMISSIBILITY PROOF:
• **Evaluation Function:** f(n) = g(n) + h(n)
  - g(n): Exact path cost from ghost's starting position to current node n (step count).
  - h(n): Heuristic estimate from node n to target: |y1 - y2| + |x1 - x2| (Manhattan Distance).
• **Admissibility Condition:** In a 4-connected grid without diagonal movement, Manhattan distance equals the true shortest physical distance in the absence of obstacles: h(n) <= h*(n). Because h(n) never overestimates true cost, A* is mathematically guaranteed to return the optimal shortest path.

a_star(self, start, goal) (Lines 319–371):
1. **Data Structures:** open_set is a binary min-heap initialized with (0, start) via heapq. came_from = {} maps each explored node to its predecessor. g_score = {start: 0} and f_score = {start: manhattan(start, goal)}.
2. **Search Loop:** Pops the node with minimal $f(n)$ via heapq.heappop(open_set). If current == goal, it traverses came_from backward, reverses the path list, and returns path[0] (the immediate next step the ghost must take).
3. **Neighbor Expansion:** Tests orthogonal neighbors [(-1,0), (1,0), (0,-1), (0,1)], accounts for wrap-around tunnel coordinates, and ignores walls. Calculates tentative_g = g_score[current] + 1. If strictly lower than existing score, records predecessor, computes $f(n) = g(n) + h(n)$, and pushes into open_set.
4. **Fallback Guarantee (Lines 357–371):** If the goal is temporarily obstructed or unreached, the algorithm iterates over legal adjacent corridors and selects the step minimizing Manhattan distance to goal. This ensures ghosts never freeze in corridors.

move_ghosts(self) (Lines 376–402) — Arcade AI Personalities:
Rather than identical behavior, each ghost executes a distinct classical targeting profile:

- **Blinky (Ghost 0, Red) & Inky (Ghost 2, Cyan) — Direct Stalkers:** Always set target = self.pacman_pos. Relentlessly pursue Pac-Man across the entire maze.
- **Pinky (Ghost 1, Magenta) — Ambush Interceptor:** Multiplies Pac-Man's movement vector by 3: target = (pacman_pos[0] + dy*3, pacman_pos[1] + dx*3). Predicts where Pac-Man is heading to cut off escape routes and head him off.
- **Clyde (Ghost 3, Yellow) — Flanker / Patrol:** Calculates Manhattan distance to Pac-Man. If distance > 6, pursues aggressively. When closer than 6 tiles, retreats to bottom corner patrol coordinates (maze_rows - 2, 1).
- **Execution:** Once the target is computed, calls next_step = self.a_star(ghost_start, target) and updates g['pos'] = next_step.

7. Collision Detection, 3-Lives System & Dialogs (Lines 403–459, 511–530)

check_collision(self) (Lines 403–407): Compares coordinates of every ghost against pacman_pos. Returns True if matching.

handle_lost_life(self) (Lines 409–437):
• Decrements self.lives -= 1.
• **If lives > 0:** Pauses the tick loop, opens the modal OUCH! CAUGHT BY A GHOST! showing remaining lives (e.g. Remaining Lives: 2 / 3). Upon key press, calls place_entities() to return Pac-Man and ghosts to starting positions. **Score, eaten pellets, and maze progress are preserved.** Returns False (game continues).
• **If lives == 0:** Triggers game_over_dialog('GAME OVER! You lost all 3 lives!', False) and returns 'QUIT' or 'MENU'.

game_over_dialog(self, message, won) (Lines 511–530):
Displays congratulations or game over banner with final score, alongside clear prompts:
  Press [Q] to Quit the Game | Press [ENTER] to Return to Main Menu

Monitors keystrokes: pressing **'Q'** returns 'QUIT' (closing the game immediately); pressing **ENTER/SPACE/ESC** returns 'MENU'.

8. The Core Engine Loop: Game.run() (Lines 460–510)

The heartbeat of the game engine is an asynchronous tick-based loop in Game.run():

- **Non-blocking Key Sampling:** Sets stdscr.nodelay(True) so getch() never blocks. If no key is detected, time.sleep(0.02) prevents CPU thrashing (~50 FPS tick rate).
- **Ghost Movement Clock:** Evaluates now - self.last_ghost_move > self.ghost_speed. When the timer expires, triggers move_ghosts() and updates the timestamp.
- **Collision & Victory Verification:** Tests check_collision(); if lives reach zero, returns the exit action. Checks remaining == 0; if all pellets are cleared, triggers victory dialog and returns.
- **Pause & Quit Keys:** Pressing **ESC** toggles self.paused. Pressing **Q** exits the match back to menu.
- **Anti-Sprint Movement Throttling (Lines 501–509):** When an arrow key is pressed, checks now - last_player_move >= player_move_delay (0.12s). If valid, moves Pac-Man and invokes curses.flushinp() to discard the operating system's hardware typematic repeat buffer. This enforces **1 key press = 1 discrete movement step**.

9. Menu System & Screen Handlers (Lines 531–718)

show_instructions_screen(stdscr) (Lines 535–573): Dedicated full-screen manual with objective, controls, and academic viva notes on A* Search, Manhattan heuristics, and cost functions.
change_difficulty_screen(stdscr, current_difficulty) (Lines 574–622): Interactive selector that adjusts ghost count (2 to 6) and ghost speed (0.50s to 0.22s).
main_menu(stdscr) (Lines 623–716): Splash screen with ASCII banner. When 'Play' returns 'QUIT' from the game over screen, it immediately breaks out of the menu loop and terminates the program cleanly.
if __name__ == '__main__': curses.wrapper(main_menu) (Lines 717–718): Program entry point.

10. Summary Cheat-Sheet for Faculty Viva

Search Algorithm	A* (A-Star) Graph Search with priority queue (heapq)	a_star() (Lines 319–371)
Heuristic Function	Manhattan Distance: $h(n) = \|y_1 - y_2\| + \|x_1 - x_2\|$ (Admissible & Consistent)	manhattan() (Lines 373–374)
Ghost Personalities	Stalking (Blinky/Inky), Intercept Ambush (Pinky), Distance-Patrol (Clyde)	move_ghosts() (Lines 376–402)
Anti-Sprint Mechanism	Throttled delta time (0.12s) + curses.flushinp() hardware buffer purge	run() (Lines 503–509)
Lives System	3 lives; resets positions on hit while preserving score, pellets, and maze	handle_lost_life() (Lines 409–437)
Quit & Menu Control	Direct [Q] key quits whole process; [ENTER] returns to arcade menu	game_over_dialog() (Lines 511–530)